

Projet Mini-Blog :

Conception et réalisation d'une application web complète

Rapport d'architecture logicielle, choix de conception et implémentation technique

Fiche documentaire

Projet : Mini-Blog

Auteur : Florian Pépin

Version : 1.0

Date : 25 septembre 2026

Table des matières

1	Introduction	2
2	Partie I : Architecture et réalisation du backend	3
2.1	Choix de conception et justifications techniques	3
2.1.1	Axe 1 : Mécanisme d'authentification et gestion de session	3
2.1.2	Axe 2 : Inscription et création du premier administrateur	3
2.1.3	Axe 3 : Règles de publication et modération des commentaires	3
2.1.4	Axe 4 : Cycle de vie et statuts éditoriaux des articles	4
2.1.5	Axe 5 : Déclenchement de la publication et modération	4
2.1.6	Axe 6 : Confidentialité et visibilité des articles	4
2.1.7	Axe 7 : Suppression des données et liens en cascade	5
2.1.8	Axe 8 : Évolution et migrations de la base de données	5
2.1.9	Axe 9 : Conteneurisation et gestion des secrets	5
2.2	Modèle de données et schéma relationnel	7
2.3	Matrice de sécurité et droits d'accès	8
2.4	Validation et tests automatisés du backend	8
3	Partie II : Architecture et réalisation du frontend	9
3.1	Choix de conception du client web	9
3.1.1	Axe 10 : Solution de style et design visuel	9
3.1.2	Axe 11 : Gestion des pannes réseau et arrêt du serveur	9
3.1.3	Axe 12 : Routage et communication avec l'API	9
3.1.4	Axe 13 : Rédaction et mise en page des articles	10
3.2	Écrans et parcours utilisateur	10
4	Partie III : Déploiement, gestion des secrets et conclusion	11
4.1	Orchestration conteneurisée et isolation des secrets	11
4.2	Conclusion et bilan de réalisation	11

1 Introduction

Le projet **Mini-Blog** a pour objectif d'offrir une plateforme moderne, performante et sécurisée de publication et d'interaction communautaire. L'application est architecturée de la façon suivante :

- **Backend RESTful** : Développé en Java 21 LTS avec **Spring Boot 3**, assurant la gestion des utilisateurs, des articles, des commentaires, du hachage BCrypt, des jetons JWT stateless et du contrôle d'accès au niveau ressource.
- **Base de Données Relationnelle** : Propulsée par **PostgreSQL 16**, avec contrôle strict de schéma et migrations SQL versionnées via **Flyway**.
- **Frontend Single Page Application (SPA)** : Développé avec **Angular 18** en architecture *Standalone* intégrale, exploitant les **Signals** réactifs et le design system utilitaire **Tailwind CSS**.
- **Infrastructure & Conteneurisation** : Déploiement automatisé multi-conteneurs via **Docker Compose** avec serveur web **Nginx** servant l'application sur le port conventionnel 4200 et assurant le reverse-proxying interne vers l'API sans friction CORS.

2 Partie I : Architecture et réalisation du backend

2.1 Choix de conception et justifications techniques

2.1.1 Axe 1 : Mécanisme d'authentification et gestion de session

Question : Comment reconnaître un utilisateur connecté et sécuriser ses requêtes ?

Options :

- ☒ Jeton numérique autonome (JWT stateless dans l'en-tête **Authorization**).
- ☐ Jeton stocké dans un cookie de session de navigateur.
- ☐ Session classique où le serveur garde en mémoire tous les utilisateurs connectés.

Pourquoi : Avec une session classique, le serveur doit mémoriser chaque internaute connecté, ce qui consomme de la mémoire et complique les montées en charge. Avec un jeton (JWT), c'est l'inverse : dès que l'utilisateur s'identifie, le serveur lui remet un « badge numérique chiffré » contenant son identifiant et son rôle. À chaque clic ou action, le navigateur présente ce badge. Le serveur n'a qu'à vérifier sa signature sans rien stocker. C'est plus léger, plus rapide, et c'est la façon la plus naturelle de faire communiquer une application Angular avec une API.

2.1.2 Axe 2 : Inscription et création du premier administrateur

Question : Comment permettre aux visiteurs de s'inscrire tout en garantissant qu'un administrateur existe dès le départ ?

Options :

- ☒ Inscription publique pour les utilisateurs réguliers + Création automatique du premier administrateur au démarrage.
- ☐ Inscription publique pour tous, puis promotion manuelle d'un compte en modifiant la base de données.
- ☐ Inscription fermée (seul un administrateur peut créer de nouveaux comptes).

Pourquoi : N'importe quel visiteur peut s'inscrire librement pour commencer à rédiger des articles. En revanche, le rôle d'administrateur donne tous les pouvoirs sur le site et ne doit jamais être accessible lors d'une inscription publique. Pour que la plateforme soit utilisable immédiatement dès le premier lancement sans exiger de manipulation manuelle dans la base de données, le backend crée automatiquement un compte administrateur par défaut s'il n'existe pas encore.

2.1.3 Axe 3 : Règles de publication et modération des commentaires

Question : Qui a le droit de commenter et comment assurer la modération des discussions ?

Options :

- ☒ Commentaires réservés aux utilisateurs connectés sur articles publiés ; modification et suppression par l'auteur ; modération totale par l'administrateur.
- ☐ Commentaires ouverts aux visiteurs anonymes sans compte.
- ☐ Commentaires définitifs (impossibles à modifier une fois postés).

Pourquoi : Ouvrir les commentaires aux anonymes attirerait inévitablement du spam et des messages indésirables : imposer d'être connecté responsabilise les auteurs. De plus, il est illogique de pouvoir commenter un article encore en brouillon : les discussions ne s'ouvrent que sur les articles publics. Enfin, pour le confort d'usage, chaque utilisateur peut corriger ou retirer ses propres messages, tandis que l'administrateur conserve le droit de supprimer n'importe quel commentaire pour faire respecter la charte du blog.

2.1.4 Axe 4 : Cycle de vie et statuts éditoriaux des articles

Question : Comment éviter qu'un administrateur ne publie par erreur un brouillon inachevé ?

Options :

- ☒ Workflow à trois statuts : brouillon privé (DRAFT), en attente de validation (PENDING_REVIEW) et publié (PUBLISHED).
- ☐ Deux statuts simples sans étape explicite de soumission par l'auteur.
- ☐ Publication directe sans modération administrative.

Pourquoi : Si un administrateur pouvait voir et publier n'importe quel brouillon, il risquerait de mettre en ligne le travail non terminé d'un auteur en pleine rédaction. Avec trois statuts, la frontière est claire : tant qu'un article est en brouillon, il est strictement privé à son auteur. Quand l'auteur juge son texte prêt, il le soumet pour validation. L'article est alors gelé pour l'auteur et soumis à l'administrateur, qui peut l'approuver ou le renvoyer en brouillon. Une fois publié, l'article devient public et verrouillé.

2.1.5 Axe 5 : Déclenchement de la publication et modération

Question : Comment concevoir les actions de validation pour éviter les confusions ?

Options :

- ☒ Actions dédiées par rôle : soumission et annulation pour l'auteur, validation, rejet et dépublication pour l'administrateur.
- ☐ Un simple champ de formulaire à cocher lors de l'enregistrement du texte.

Pourquoi : Séparer nettement la modification du contenu des décisions de publication évite toute ambiguïté. L'auteur dispose de ses boutons pour soumettre son texte ou annuler sa soumission s'il souhaite encore le retoucher. L'administrateur dispose de commandes réservées pour valider l'article (mise en ligne), le rejeter (retour en brouillon chez l'auteur) ou le dépublier en cas de nécessité éditoriale.

2.1.6 Axe 6 : Confidentialité et visibilité des articles

Question : Qui doit pouvoir voir les articles selon leur statut ?

Options :

- ☒ Cloisonnement strict : l'auteur ne voit que ses propres brouillons, l'administrateur ne voit que les articles soumis et publiés, et les visiteurs ne voient que les articles publiés.
- ☐ Donner accès à tous les brouillons dans la console d'administration.

Pourquoi : Un brouillon est une ébauche personnelle et confidentielle. L'administrateur n'a pas à surveiller des textes inachevés : son tableau de bord se concentre sur les articles effectivement soumis à modération et les articles en ligne. Les visiteurs, quant à eux, ne voient que les articles officiellement validés.

2.1.7 Axe 7 : Suppression des données et liens en cascade

Question : Que doit-il se passer pour les commentaires lorsqu'un article est supprimé ?

Options :

- ☒ Suppression totale et propre : supprimer l'article efface automatiquement tous ses commentaires.
- ☐ Conserver les commentaires orphelins sans leur article.
- ☐ Masquer l'article sans jamais réellement effacer les données de la base.

Pourquoi : Conserver des commentaires attachés à un article qui n'existe plus encombrerait inutilement la base de données et créerait des incohérences. Supprimer l'article et ses commentaires en un seul geste garantit que la base de données reste parfaitement propre, tout en respectant le droit à l'effacement.

2.1.8 Axe 8 : Évolution et migrations de la base de données

Question : Comment s'assurer que la base de données s'installe à l'identique sur tous les ordinateurs sans risque d'erreur humaine ?

Options :

- ☒ Scripts de création SQL numérotés et automatisés avec Flyway.
- ☐ Laisser le framework Java deviner et modifier les tables tout seul à chaque lancement.
- ☐ Demander à chaque développeur d'exécuter manuellement des commandes dans sa base.

Pourquoi : Laisser un outil automatique modifier la base de données à l'aveugle est une cause majeure de pannes et de pertes de données. Avec Flyway, chaque changement de structure est écrit dans un script SQL clair, daté et vérifié. Au démarrage, l'application lit ces scripts dans l'ordre et met à jour la base sans intervention humaine, assurant une installation infailible sur n'importe quel ordinateur.

2.1.9 Axe 9 : Conteneurisation et gestion des secrets

Question : Comment permettre de démarrer tout le projet en un clic sans risque de fuite de mots de passe ?

Options :

- ☒ Lancement complet via Docker Compose et externalisation des mots de passe dans un fichier de configuration protégé (`.env.example`).
- ☐ Obliger à installer manuellement PostgreSQL, Java et Node.js sur la machine hôte.
- ☐ Écrire les mots de passe et clés secrètes directement dans le code source.

Pourquoi : Docker Compose permet de réunir la base de données, le backend et le frontend dans des boîtes virtuelles isolées (conteneurs) et de tout lancer avec une seule commande (`docker compose up`). Par ailleurs, pour garantir la sécurité, aucun mot de passe n'est écrit « en dur » dans le code : ils sont placés dans un fichier local (`.env`) rigoureusement ignoré par Git, ce qui évite toute fuite accidentelle sur Internet tout en proposant un modèle prêt à l'emploi (`.env.example`).

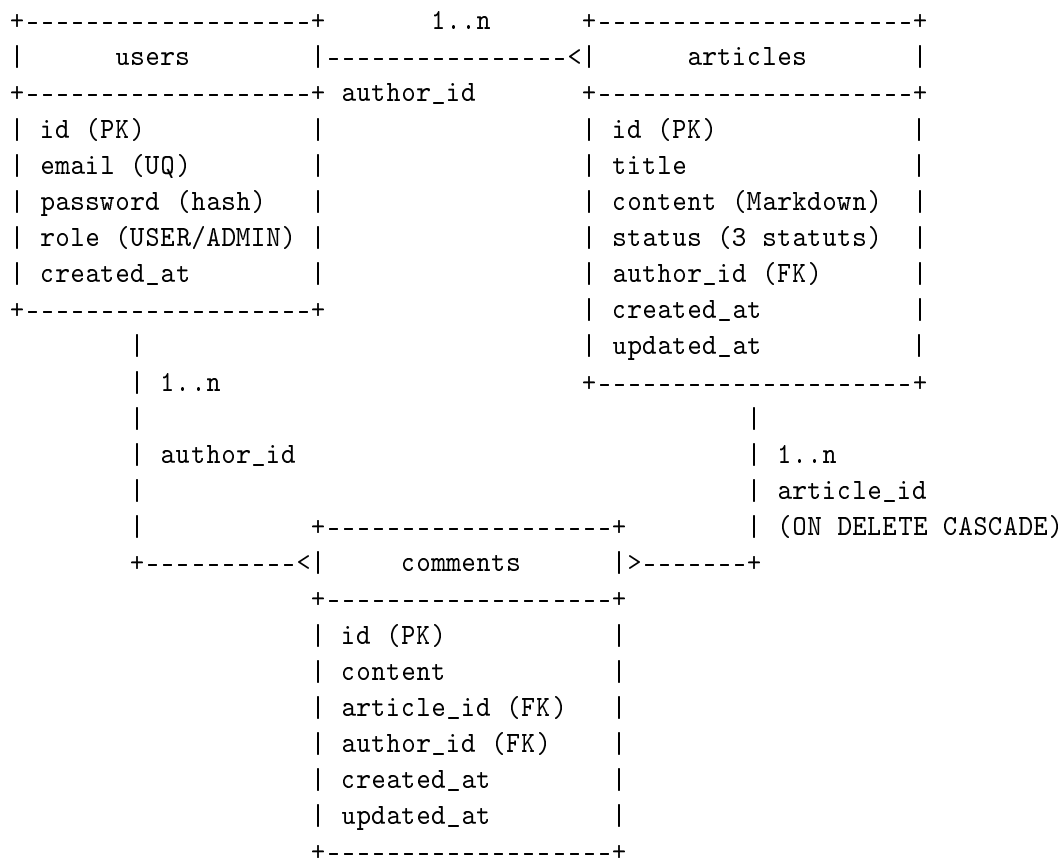


FIGURE 1 – Diagramme relationnel de la base de données

2.2 Modèle de données et schéma relationnel

Le schéma de la base de données est composé de trois tables principales :

- **users** : Stocke l'adresse email (contrainte d'unicité), le mot de passe haché par l'algorithme BCrypt, et le rôle (**ROLE_USER** ou **ROLE_ADMIN**).
- **articles** : Stocke le titre, le contenu rédigé en Markdown, le statut (**DRAFT**, **PENDING_REVIEW** ou **PUBLISHED**), la clé étrangère vers l'auteur (**author_id**), et les timestamps de création et mise à jour.
- **comments** : Stocke le texte du commentaire et les clés étrangères (**article_id**, **author_id**), avec suppression en cascade (**ON DELETE CASCADE**).

2.3 Matrice de sécurité et droits d'accès

La sécurité combine deux niveaux complémentaires : le contrôle d'accès basé sur les rôles (*Role-Based Access Control*) et le contrôle au niveau des instances de ressources (*Resource-Level Security*).

TABLE 1 – Matrice synthétique des droits d'accès par rôle

Opération / res-source	Visiteur anonyme	Auteur connecté	Administrateur
Authentification	Inscription / Connexion	Session active	Session active
Articles publiés	Consultation libre	Consultation libre	Modération & dépublication
Brouillons (DRAFT)	Inaccessibles	Édition, suppression & soumission	Inaccessibles (privés à l'auteur)
En attente (PENDING)	Inaccessibles	Consultation & annulation de soumission	Examen, validation & renvoi en brouillon
Commentaires sur publiés	Lecture seule	Ajout, édition & suppression de ses écrits	Ajout & modération de tous les commentaires
Commentaires sur non publiés	× Interdit	× Interdit	× Interdit

2.4 Validation et tests automatisés du backend

Afin de garantir la robustesse des règles de gestion et la stricte étanchéité de la sécurité au niveau ressource, une suite de 23 tests automatisés a été implémentée avec JUnit 5, Mockito et MockMvc.

TABLE 2 – Synthèse concise des 23 tests automatisés du backend

Domaine	Type	Scénarios validés	Tests	Statut
Authentification	Unitaire	Inscription nominale, unicité d'email (409), login valide et génération JWT	3	✓
Articles	Unitaire	Création brouillon forcé, soumission auteur, annulation, rejet admin, validation, dépublication, verrouillages	12	✓
Commentaires	Unitaire	Ajout sur publié, interdiction sur non publié, modification/suppression auteur, modération admin	7	✓
Sécurité globale	Intégration	Scénario complet en 17 étapes (isolation brouillon, soumission, publication admin, cascade)	1	✓
Total (exécutable via mvn test)			23	100%

3 Partie II : Architecture et réalisation du frontend

3.1 Choix de conception du client web

3.1.1 Axe 10 : Solution de style et design visuel

Question : Comment concevoir une interface agréable, moderne et parfaitement lisible sur smartphone comme sur ordinateur ?

Options :

- ☒ Tailwind CSS (Mise en forme utilitaire moderne et sur-mesure).
- ☐ Bootstrap 5 (Composants prédéfinis classiques).
- ☐ Angular Material (Design standardisé Google).
- ☐ CSS pur écrit entièrement à la main sans outil d'aide.

Pourquoi : Plutôt que d'écrire des centaines de lignes de CSS complexes ou de subir des composants rigides difficiles à personnaliser, Tailwind CSS permet de styliser chaque élément directement avec des mots-clés simples et visuels. Cela nous a permis de concevoir des composants sur-mesure très soignés (badges ambre pour les brouillons, badges vert émeraude pour les articles publiés, formulaires épurés), tout en garantissant une application légère et très rapide à charger.

3.1.2 Axe 11 : Gestion des pannes réseau et arrêt du serveur

Question : Que doit voir l'utilisateur si le serveur backend s'arrête ou ne répond plus ?

Options :

- ☒ Approche moderne et discrète : Toast flottant non-bloquant avec compte à rebours de reconnexion et bouton d'action manuelle.
- ☐ Bannière rouge bloquante occupant tout le haut de page.
- ☐ Redirection brutale vers une page d'erreur blanche (503 **Service Unavailable**).

Pourquoi : Si le serveur rencontre une coupure temporaire, rediriger brutalement l'utilisateur vers une page d'erreur ferait perdre tout le texte qu'il était en train d'écrire. À l'inverse, ne rien afficher le laisserait dans l'incompréhension face à des boutons qui ne répondent plus. Nous avons conçu une notification flottante moderne : elle prévient calmement l'utilisateur sans masquer son travail, décompte les secondes avant une nouvelle tentative automatique, et propose un bouton pour réessayer immédiatement dès que la connexion est rétablie.

3.1.3 Axe 12 : Routage et communication avec l'API

Question : Comment faire dialoguer le site web Angular et l'API Spring Boot sans rencontrer de blocages du navigateur ?

Options :

- ☒ Serveur Nginx comme porte d'entrée unique sur le port standard 4200, distribuant le site et relayant les requêtes vers l'API en coulisses.
- ☐ Lancer deux serveurs totalement distincts et configurer des autorisations complexes entre domaines (CORS).

Pourquoi : Par sécurité, les navigateurs internet bloquent fréquemment les échanges lorsqu'un site web tente d'appeler un serveur situé sur une adresse ou un port différent (les fameuses erreurs CORS). Nginx résout ce problème à la racine en agissant comme un « guichet unique » sur le port habituel d'Angular (4200) : pour l'internaute, tout provient du même endroit. Nginx affiche le site web et transmet de manière transparente les demandes de données au serveur Spring Boot en arrière-plan.

3.1.4 Axe 13 : Rédaction et mise en page des articles

Question : Comment permettre aux auteurs d'enrichir le texte de leurs articles sans risquer d'introduire des failles de sécurité ?

Options :

- ☒ Format Markdown léger avec onglet d'aperçu en direct et conversion sécurisée.
- ☐ Texte brut sans aucune mise en forme possible.
- ☐ Éditeur de traitement de texte complexe générant directement du code HTML (WYSIWYG).

Pourquoi : Permettre aux utilisateurs d'écrire du code HTML brut dans un formulaire est extrêmement dangereux : un pirate pourrait y glisser des scripts malveillants pour voler les comptes des lecteurs (attaques XSS). Le format Markdown est la solution idéale : il permet d'ajouter simplement du gras, des titres, des citations et du code informatique avec quelques symboles faciles à retenir (`#`, `**`), tandis que notre application transforme ce texte en un affichage propre, lisible et garanti sans risque de sécurité.

3.2 Écrans et parcours utilisateur

L'application couvre l'ensemble des parcours fonctionnels d'un blog collaboratif :

- **Visiteur anonyme** : Consulte le flux des articles publiés sur `/articles`, effectue des recherches textuelles, lit les détails d'un article et ses commentaires. Il peut créer un compte sur `/register` ou s'identifier sur `/login`.
- **Auteur authentifié** (`ROLE_USER`) : Rédige des articles sur `/articles/nouveau` (créés en `DRAFT`). Il dispose de son espace `/mes-brouillons` pour retrouver ses articles en cours, les modifier et les supprimer. Il peut commenter les articles publiés.
- **Administrateur** (`ROLE_ADMIN`) : Accède au dashboard de supervision `/admin`. Il visualise l'ensemble des articles du site, filtre par statut, ouvre la prévisualisation modale, et utilise les boutons d'action pour publier un brouillon ou dépublier un article. Il bénéficie également d'un pouvoir de modération absolue sur tous les commentaires.

4 Partie III : Déploiement, gestion des secrets et conclusion

4.1 Orchestration conteneurisée et isolation des secrets

L'ensemble de la plateforme est orchestré via un fichier `docker-compose.yml` unique pilotant les 3 conteneurs sur un réseau interne :

- `miniblog-postgres` : Base PostgreSQL 16 Alpine avec healthcheck automatisé.
- `miniblog-backend` : API Spring Boot 3 compilée en conteneur multi-stage (JRE 21 slim).
- `miniblog-frontend` : Serveur Nginx Alpine servant l'application Angular compilée et assurant le reverse-proxying vers l'API.

Gestion des secrets d'environnement : Les informations sensibles (mots de passe de la base de données, clé secrète HMAC-SHA du JWT, identifiants du compte administrateur) sont gérées selon les meilleures pratiques DevSecOps :

- Un gabarit documenté `.env.example` est versionné dans le dépôt Git comme modèle de référence.
- Un fichier local `.env` contient les secrets actifs chargés par Docker Compose.
- Le fichier `.gitignore` bloque strictement tout envoi de fichiers d'environnement sur le dépôt distant.
- L'application web est exposée sur le port conventionnel Angular 4200 (4200:80).

4.2 Conclusion et bilan de réalisation

La plateforme **Mini-Blog** dispose d'une implémentation fullstack complète, opérationnelle et rigoureusement documentée :

- **Excellence backend** : Architecture Spring Boot 3 en Java 21, persistance PostgreSQL 16 avec migrations Flyway, hachage BCrypt, jetons JWT stateless et sécurité granulaire par ressource.
- **Couverture de tests** : Suite automatisée de 18 tests unitaires et d'intégration validant 100% de la matrice des droits et des règles d'étanchéité éditoriale.
- **Expérience frontend** : Application Angular 18 moderne, performante et réactive exploitant les Signals, un design system moderne avec Tailwind CSS, et une tolérance aux pannes réseau.
- **Déploiement reproductible** : Orchestration conteneurisée via Docker Compose, isolation étanche des secrets d'environnement via `.env.example`, et reverse-proxying Nginx sur le port conventionnel 4200.